



گزارش پروژه بینایی کامپیوتر

استاد درس

دکتر محمدی

دانشجویان:

آرمان حیدری

شایان موسوی نیا

فهرست

۳.....	مقدمه
۳.....	Siamese
۵.....	توابع pre-processing
۷.....	سایر ایده های پیاده سازی شده
۸.....	منابع

مقدمه

ایده اولیه، مقایسه طرح و کاشی بدون استفاده از مدل های **deep learning** بود. بدین صورت عمل می کردیم که بعد از **extract** کاشی و از عکس کاشی پایه و تبدیل مقیاس و اندازه طرح به اندازه کاشی **extract** شده، سعی داشتیم که با انجام یکسری عملیات، این ۲ تصویر را از همدیگر کم کنیم و انتظار داشتیم بخش اصلی که باقی میماند، صرفاً ترک های ما باشند.

چالش اصلی این موضوع، این بود که طرح و کاشی دقیقاً روی همدیگر قرار نمی گرفتند. برای برطرف کردن این موضوع، با استفاده از پیدا کردن **key point** های هر ۲ تصاویر و انطباق این **key point** ها بر روی همدیگر، تا حد خوبی توانستیم این مشکل را برطرف کنیم و سپس بعد از **binary** کردن تصویر به وسیله الگوریتم **adaptive threshold** عکس را باینری کردیم. در انتها پس از ۵ باز استفاده از عملگر **opening** و **closing** بر روی کاشی، آن ۲ رو از هم کردیم.

در این مرحله، حاصل ما دارای ترک و یکسری از مرز های طرح های ما بود که موقع کم کردن طرح و کاشی، باقی مانده اند. برای حل این موضوع، هر پیکسلی که در بالا بعد از اختلاف کاشی و طرح از همدیگر باقی ماند را در همسایگی ۲ پیکسل نسبت به طرحی که باینری شده است بررسی کردیم.

بدین گونه عمل کردیم که اگر در همسایگی به اندازه ۲ پیکسل فاصله، در طرح ما پیکسلی سفید داشتیم، برداشتمان این بود که این پیکسل، ترک نیست و مرزی از طرح است که موقع تفریق باقی مانده است. حال هر مقداری که باقی ماند را به عنوان ترک به خروجی می دهیم.

Siamese

الگوریتم اصلی که ما برای تشخیص ترک استفاده کردیم، **siamese** بود که این شبکه عصبی کلاسی از معماری شبکه های عصبی است که شامل دو یا چند زیر شبکه یکسان است. "یکسان" در اینجا به این معنی است که آنها پیکربندی یکسانی با پارامترها و وزن های یکسان دارند. برای استفاده از این شبکه، ما نیاز به تهیه ۳ گروه ورودی برای آموزش شبکه داشتیم.

- گروه اول، عکس های طرح ها بود که باید تعدادی **preprocess** روی آن انجام می دادیم تا آماده **fit** کردن به شبکه باشد. چون برای مثال تعدادی از طرح ها دچار چرخش نسبت به عکس اصلی شده بودند، و با تابع **find_rotate** آن ها را به جهت مناسب در آوردیم. همچنین ابعادشان 5000×5000 بود که باید به

ابعاد عکس اصلی درمی آمد. هیستوگرام برخی از این طرح ها خیلی contrast بالایی داشت و عمدتا در یک محل جمع شده بود، که این موضوع را با اعمال histogram equalization روی آن همدل کردیم.

- گروه دوم، عکس هایی کاشی های ما بود که بعد از اعمال pre process بر روی آن ها در لیست دوم ما قرار می گرفتند. این pre process ها شامل histogram matching بر اساس طرح و histogram Equalization و چندین مورد دیگر بود. این مراحل به صورت دقیق تر در بخش توضیح توابع آورده شده است.

- گروه سوم label های ما بودند که مشخص میکرد ایا در عکس کاشی متناظر با این index در لیست ما، ترکی داریم یا خیر.

ما این شبکه را یک بار در حالتی که ورودی ها ۳ کاناله باشد، یعنی رنگی باشد اجرا کردیم که در آن حالت histogram equalization انجام ندادیم چون اگر در هر کانال جداگانه میزدیم تصویر به هم میریخت. البته histogram matching را همواره انجام دادیم که تصویر و طرح هرچه بیشتر به هم نزدیک شوند.

ایده اصلی برای استفاده از این شبکه این بود که توانایی یادگیری برای مقایسه شباهت های تو تصویر ورودی را داشت که میتوانستیم با مقایسه قسمتی از طرح و کاشی، تشخیص دهیم که آیا تفاوت قابل ملاحظه ای بین این ۲ تصویر وجود دارد و یا نه که اصولا انتظار داشتیم این تفاوت به خاطر وجود ترک در کاشی و نبود اثر آن در طرح اصلی باشد. در ابتدا ما شروع به بخش بندی عکس های کاشی ها و طرح هایشان به قطعات کوچکتر می شویم. در مرحله اول سائز پنجره ای که برای این کار در نظر گرفتیم ۱۲۰*۱۲۰ است. به از آموزش مدل برای روی تمامی این داده ها، ما نیاز هست به همین فرمتی که برای سائز ۱۲۰*۱۲۰ در نظر گرفتیم، مدل های جدیدی را بر اساس سائز های زیر نیز آموزش دهیم:

- 60*60
- 30*30
- 15*15
- 5*5

بعد از اتمام آموزش شروع به ساخت تابع پیش بینی خود کردیم. به این صورت عمل کردیم که یک تصویر و طرح آن را به مدلی که سائز آن عکس های ۱۲۰*۱۲۰ بود میدهیم و هر جا که پیش بینی کرد خروجی ما برابر با ۱ باشد به این معنا هست که تشخیص داده در آن ترکی وجود دارد. پس در این مرحله ما به پنجره های ۱۲۰*۱۲۰ ای میرسیم که مدل ما پیش بینی میکند در آن ترک وجود دارد و یا خیر. سپس تمامی این پنجره هایی را که به دست آوردیم را به مدلی با پنجره سائز ۶۰*۶۰ میدهیم. بدین گونه که پنجره ۱۲۰*۱۲۰ ای که در آن پیش بینی شده است که ترک وجود دارد را به ۴ پنجره ۶۰*۶۰ تبدیل میکنیم و مدل با پنجره ۶۰*۶۰ را روی تمامی این پنجره های جدید دوباره برای پیش بینی اجرا میکنیم. مانند فرایند بالا به یک سری پنجره ۶۰ در ۶۰ خواهیم رسید که مدل ما پیش بینی کرده در آنها ترک وجود دارد. جال این پنجره های ۶۰ در ۶۰ را به ۴ تصویر ۳۰ در ۳۰ تبدیل خواهیم کرد و به مدلی که بر اساس پنجره

ای با این سائز آموزش دیده است میدهم. این فرایند را تا آخر با باقی مدلها ادامه میدهم تا اینکه به پنجره های ۵*۵ رسیده باشیم که مدل آنها را به عنوان ترک شناسایی کرده است.

تمامی این پنجره های ۵*۵ را به عنوان پارتی از ترک نمایش خواهیم داد.

ما ۲ روش برای آموزش مدل هایی مه در بالا توضیح دادیم پیش گرفتیم:

- استفاده از وزن های resnet و imagenet و fine tune کردن روی این وزن ها

- آموزش پایه بر روی مدل بدون هیچ وزن اولیه

بخش پایه Siamese شامل یک سری لایه conv و dense است که به راحتی می توان از وزن های آماده روی آن استفاده کرد. با استفاده از وزن های گفته شده در بالا ما برای مدل با پنجره ۱۲۰*۱۲۰ توانستیم به f1 score برابر با ۹.۸ درصد در داده های test برسیم و این مقدار برای حالتی که از هیچ وزن آماده ای استفاده نمیکنیم برابر با ۴.۴ درصد بود.

نکته قابل ارزنده این است که در حالت وزن های imagenet مدل ما توانسته است به recall ۱ برسد که این موضوع بیانگر این است که با وجود اینکه f1 score بالایی را نتوانسته است کسب کند ولی تعداد FN های ما برابر با ۰ است. این نشان دهنده این است که هیچ ترکی نبوده است که مدل ما آن را پیدا نکرده باشد!

مدل استفاده شده هم نسبتا ساده است. بعد از استفاده از ResNet ۵۰، ما از ۴ لایه dense با سائز های ۵۱۲، ۲۵۶، ۲۵۶ و ۱۰ استفاده کردیم که این مدل پایه ای است که در جفت شبکه عصبی که در Siamese استفاده کردیم مورد بهره قرار گرفته است.

چالش اصلی این مدل، imbalance بودن برچسب های موجود آن بود که در حالت کلی، نسب تعداد برچسب های ۰ (غیر ترک) به تعداد ۱ ها (ترک) در حدود ۴۰ بود. برای برطرف کردن این موضوع ما به برچسب هایی که داشتیم، وزن هایی اختصاص دادیم که مقدار مناسب های را با استفاده از genetic algorithm محاسبه کردیم. به این صورت که تابع fit برای آموزش مدل را به عنوان تابع reference برای این GA قرار دادیم و برای آن بازه ای بین نسبت label های ۰ و ۱ که بر ۲ تقسیم کردیم و نسب همین label ها که بر ۱۰ ضرب کردیم در نظر گرفتیم که بعد از به اتمام رسیدن آن، بهترین نتیجه برای این hyper parameter برابر با ۹.۸ بود.

توابع pre-processing

Apply_threshold:

هدف از این تابع باینری کردن عکس اصلی بود که با کمک آن بتوانیم بک گراند را حذف کنیم. آن را با روش های مختلفی مثل adaptive threshold و threshold و otsu امتحان کردیم که سومی از همه بهتر بود و تقریبا کاشی را از زمینه جدا می کرد البته مشکلاتی به وجود می آورد که با تابع بعدی آن را حل کردیم.

Morphology:

میدانیم عملگر **opening** برای حذف نویز ها به ما کمک میکند، پس با اعمال یک **opening** نسبتاً بزرگ (ابعاد **kernel** را با چند نمونه آزمون و خطا کردیم و نتیجه باینری کردن را **visualize** کردیم، که کدش هم کامنت شده است، سعی میکنیم موارد روشنی که خارج از مستطیل کاشی بوده اند را بدون این که ابعاد کاشی تغییری یابد حذف کنیم. بع هم از یک مورفولوژی **closing** استفاده میکنیم که بخش های خالی درون کاشی را سفید کند و تمام کاشی خیلی عالی از یک گراند جدا می شود.

توضیح: به جای مجموع این دو تابع بالا، همچنین تلاش کردیم از **find contours** استفاده کنیم و بیشترین مساحت آن را به عنوان کاشی بگیریم. همینطور سعی کردیم با **canny** کردن عکس ها و پیدا کردن گوشه ها با **harris** هم کاشی را بیابیم اما بین این دو روش و روشی که بالا توضیح دادیم، روش پیاده سازی شده دقت بهتری داشت.

Crop_image:

در این تابع اضافات عکس را میبریم تا فقط کاشی بماند. در واقع نقاط متناظر سیاه در عکس باینری به دست آمده با توابع قبلی را، از عکسمان حذف میکنیم. نکته مهم این است که نقطه مینیمم **X** و **Y** (بالا سمت چپ مستطیل کاشی) را باید پیدا کنیم و همه **label** هایمان را که در **json** هستند هم به همان اندازه شیفت بدهیم تا **X** و **Y** آن ها متناسب با تصویر عوض شود و همچنان دور ترک باشد.

Histogram_matching:

همانطور که قبل تر توضیح دادم به این تابع برای تناظر بهتر بین طرح و هر عکسش نیاز داریم و عکس اصلی را با هیستوگرام طرحش تحقیق میدهد به امید مقایسه راحت تر آن ها.

Extract_pattern:

با این تابع میتوانیم برای هر عکسی، **pattern** متناظر آن را از **json** متناظرش استخراج کنیم.. این تابع برای برخی کدهای بعدی توسعه داده شد.

Find_label:

این تابع یکی از مهم ترین تابع هایی است که مورد استفاده قرار گرفته و وظیفه آن این است که مشخص کند آیا در پنجره های مورد بررسی قرار دادیم، ترکی وجود دارد و یا خیر. برای این موضوع نیز ما پنجره را به عنوان یک **polygon** در اوریم و از طرفی دیگر نیز همین کار را برای برجسب ترک ها نیز انجام میدهیم. در اخر سر با اشتراک گیری از این ۲ **polygon** مشخص میکنیم که آیا این ۲ با هم اشتراکی داشته اند و یا خیر.

Window_sliding:

در این تابع حلقه ای به طول و عرض اندازه **window size** میزنیم (که هایپر پارامتر مهمی است و بارها برای آموزش آن را عوض کردیم) و عکس را به قطعات مساوی تقسیم میکنیم تا پنجره های پیشنهادی ما باشند. این روش را برای عکس اصلی و **pattern** که پیش پردازش هایش انجام شده است انجام میدهیم. همچنین با کمک تابع قبلی و با پیدا کردن **polygon** مربوط به **label** این عکس (اطراف ترک)، برای هر جفت متناظر از عکس و طرح برجسب گذاری میکنیم. در واقع ۳ لیستی که این تابع برمیگرداند، پنجره های عکس، پنجره های طرح و ۰ یا ۱ به معنی نبودن یا بودن ترک در آن هاست.

:save_image:

پس از همه این پیش پردازش ها که عکس اصلی و ابعادش و متناظر با آن `json` اش را و حتی طرحش را تغییر میدهد، با ذخیره سازی به اجرای برخی شبکه ها مثل `yolo` کمک شایانی می کند. البته این تابع در ورژن `siamese` نیست چون نیازش نداشتیم و مستقیماً در لیست هایی ذخیره می کردیم.

Pre_processing:

به نوعی `main` این قسمت است و همه توابع به ترتیب مناسب صدا می شوند.

rotate_finder:

یکی از چالش های بررسی عکس کاشی با طرح آن، تفاوت داشتند زاویه این ۲ تصویر در خیلی از حالات با هم دیگر بود. برای برطرف کردن این مورد، نیاز بود بتوانیم طرح را در زاویه مناسبی چرخش دهیم که با کاشی، جهت یکسانی داشته باشد. برای این کار از `"image hash.average hash"` استفاده کردی که تصویر را `hash` میکند و عکس خروجی از این تابع، به صورت یک تصویر 8×8 خواهد بود. هر پیکسل این تصویر، `hash` شده یک سری پیکسل همسایه در کنار هم است. حال با `hash` کردن هم طرح و هم تصویر، میتوانیم به مقیاس عددی مناسبی برای مقایسه ۲ تصویر برسیم. ما طرح را در ۴ جهت ۰، ۹۰، ۱۸۰ و ۲۷۰ چرخانیدیم و `hash` شده آن را با عکس کاشی خود مقایسه کردیم. بعد از محاسبه تفاوت این ۴ `hash` با `hash` کاشی اصلی، هر کدام که کمتر بود را به عنوان زاویه چرخش مناسب برای طرحی که داریم در نظر میگیریم.

سایر ایده های پیاده سازی شده

ایده دیگری که مورد استفاده قرار گرفت، استفاده از شبکه های آماده مانند `yolo v` و `RetinaNet` برای پیدا کردن ترک های موجود است. استفاده از این شبکه ها به ما اجازه میدهد که بتوانیم از مدل هایی استفاده کنیم که دارای وزن های `pretrained` مناسبی برای آموزش هستند. تنها موردی که برای اجرای این ۲ الگوریتم نیاز داریم، تهیه مجموعه داده مناسب برای آموزش است.

در بخش `Siamese` ما با کمک تابع `pre process`، کاشی ها را از فرم اولیه آن جدا کردیم. حال برای اینکه بتوانیم این کاشی ها را به شبکه هایی که در بالاتر اشاره کرده ایم بدهیم، نیاز به تغییر فرمت `label` ها داشتیم. `Label` هایی که در داده های اولیه به ما داده شده بود به صورت یک سری نقاط بودند که با وصل کردن آنها به هم یک `polygon` دست میشد که برچسب ما را نشان میداد. شبکه های `yolo` و `retina` برچسب هایی که به عنوان ورودی دریافت میکنند به فرمت `box` است.

برای برطرف کردن این موضوع، ما هر `label` ای که به عنوان ترک داشتیم را، مقادیر `min_x`، `mix_y` و همینطور `max_x` و `max_y` را برای آن مجموعه نقاط موجود در آن برچسب پیدا میکنیم و با این مقادیر، ۴ نقطه درست میکنیم که `box` های ما را مشخص میکند.

نکته مهم این است که فرمت برچسب های مورد استفاده در `yolo` در ابتدا با شماره آن برچسب شروع میشود که نشان دهنده این است که مربوط به کدام گروه است در ادامه ۲ نقطه `x,y` برای مرکز این `box` در باید بنویسم و در آخر سر

نیز ارتفاع و عرض این box نیز باید مشخص شده باشند. البته این موضوع را هم باید خاطر نشان کرد که تمامی این مقادیر باید normalize شوند و برای این موضوع بر طول و یا عرض تصویر تقسیم میشوند.

بعد از محاسبه و ذخیره برچسب ها جدید، مدل های بالا را شروع به آموزش دادیم. برای yolo v ما از مدل large ان با batch size برابر با ۴ و همین طور سائز $1280 * 1280$ استفاده کردیم که مقدار mAP ۵۰ ما برابر با ۳.۴ درصد شد و با همین hyper parameter ها برای retinaNet توانستیم به mAP ۵۰ برابر با ۱۳ درصد برسیم.

همچنین قصد پیاده سازی شبکه u-net را داشتیم که برای آن باید یک تصویر ورودی آماده میکردیم. در نوتبوک مربوطه مشخص است که تصاویر را بعد از preprocess ذخیره کرده ایم و سعی کردیم طرح را به نحوی از تصویر اصلی حذف کنیم به این امید که فقط ترک و کمی نویز بماند اما چون دقت خوبی نداشت، به سراغ ادامه کار و شبکه U-net رفتیم.

در انتها، بهترین الگوریتم پیاده سازی شده مربوط به استفاده از Siamese است و با توجه به معماری که مانند SSD دارد، ما توانستیم به قطعه بندی تصویر به پنجره هایی با سائز های متفاوت به دقت بهتری نسب به باقی الگوریتم ها برسیم. همینطور مزیت دیگر آن نسبت به yolo و retinaNet، سرعت بالاتر آن و در کنار برچسب دقیق تر بر روی ترک به نسب این ۲ الگوریتمی است که برچسب های آنها به صورت box است.

منابع

- <https://github.com/yhenon/pytorch-retinanet>
- <https://github.com/ultralytics/yolov5>
- https://keras.io/examples/vision/siamese_contrastive/